

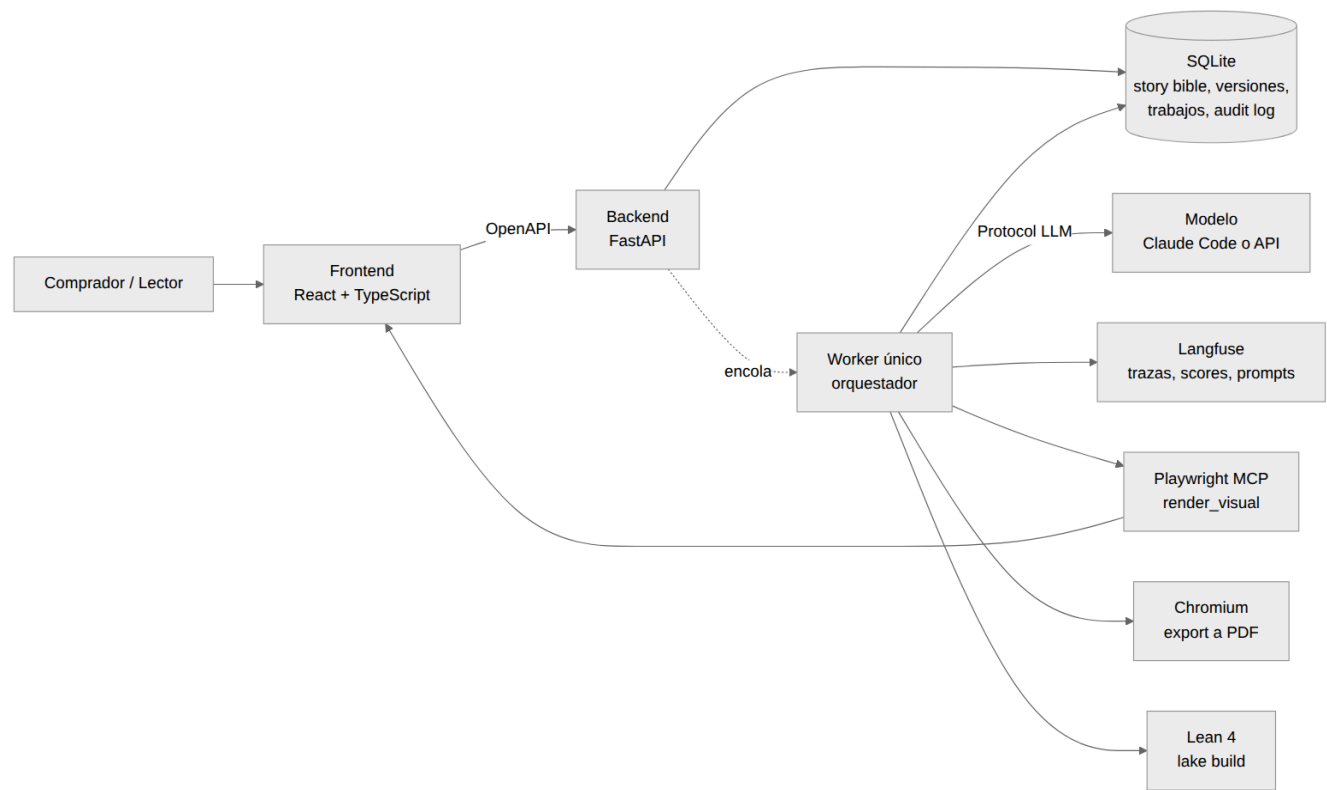
Anexo — Diagramas de arquitectura detallados

Sep 25, 2026 · @Bruno Cruz

Diagramas de la arquitectura del harness de generación de novelas, reconstruidos a partir de las decisiones de diseño del proyecto. Antes de presentarlos, contástalos con `docs/architecture.md` y el código: los nombres de componentes y estados deben coincidir exactamente.

A.1 · Vista del sistema

El backend en FastAPI orquesta la generación con una máquina de estados propia; un único worker consume los trabajos de SQLite, y el modelo, Langfuse, Playwright y Lean son dependencias externas detrás de interfaces propias.

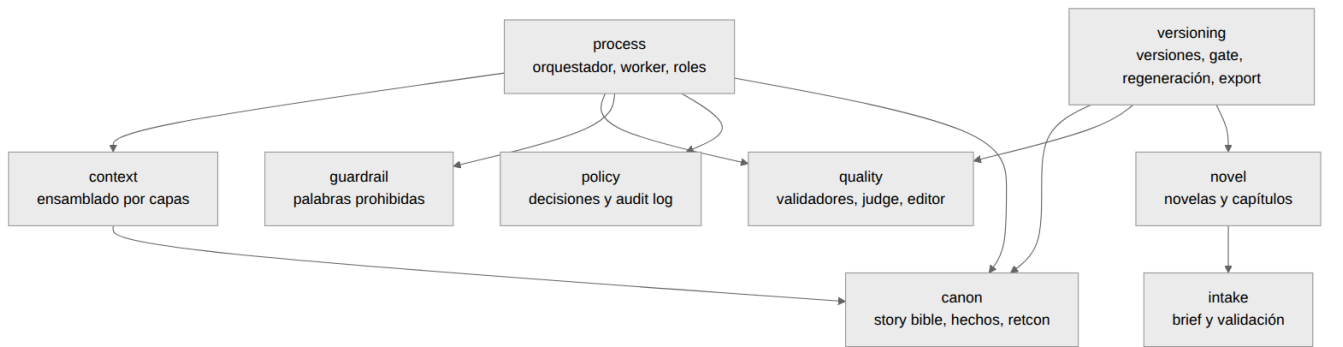


Componente	Decisión	Motivo
Orquestador propio	Máquina de estados en FastAPI, no un framework de agentes	El proceso es conocido: control del flujo, coste acotado y correspondencia con TLA+

Componente	Decisión	Motivo
Worker único sobre SQLite	Cola de trabajos en una tabla, sin Redis ni Celery	Sobrevive a reinicios y permite reanudar desde checkpoint
Proveedor del modelo	Protocol con dos implementaciones: API y Claude Code	Cambiar de proveedor es configuración, no código
Contrato OpenAPI	Escrito antes del código	Frontend y backend se construyeron en paralelo

A.2 · Mapa de features del backend

El backend se organiza en nueve features verticales más `commons/` ; cada feature es dueña de sus clases y solo importa el `service.py` de otras, nunca su repositorio.



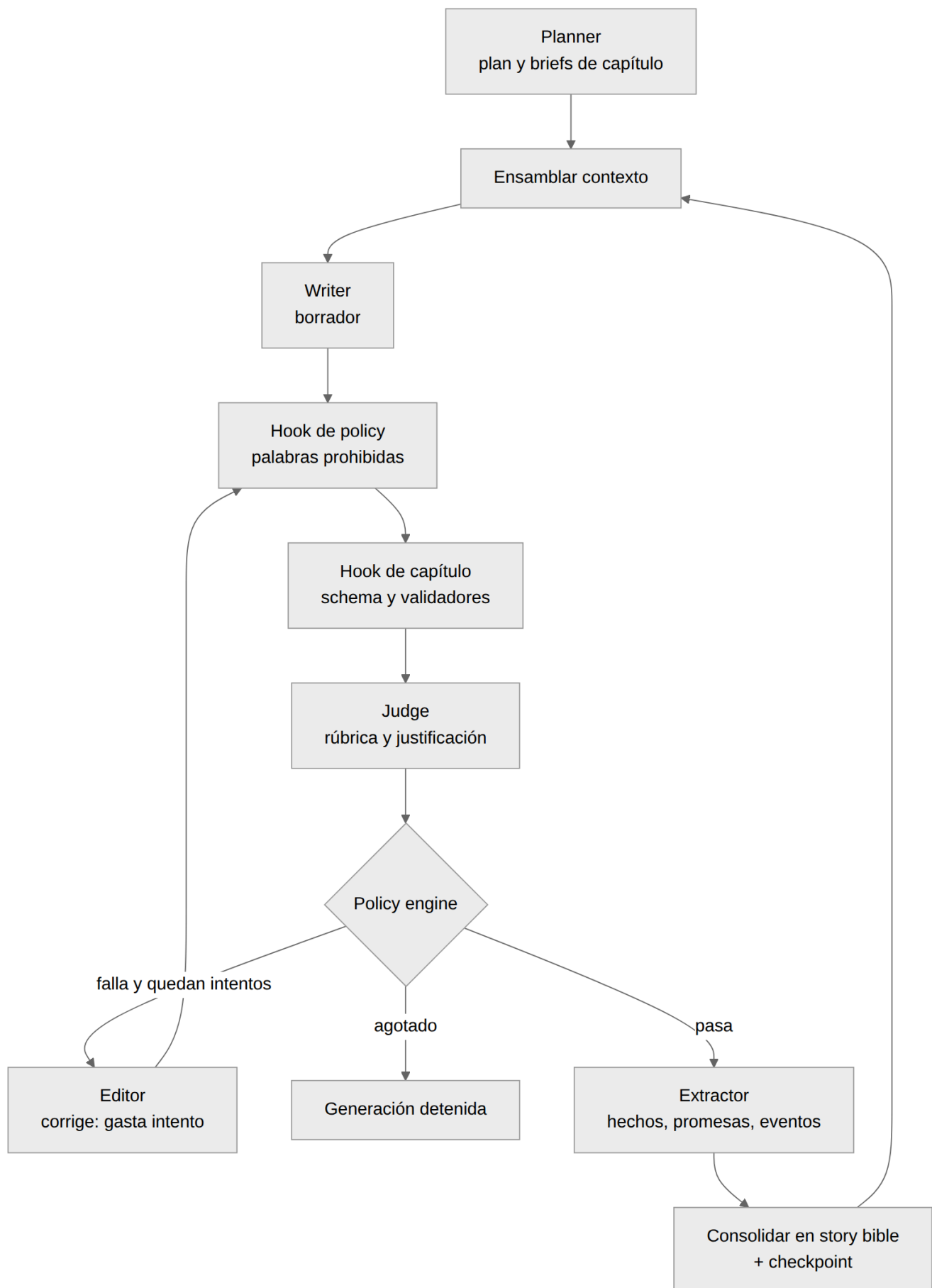
Feature	Qué contiene
intake	Brief parcial y completo, detección de datos faltantes y contradicciones, saneamiento del texto libre
novel	Novelas y capítulos
canon	Story bible con vigencia por versión, hechos, promesas, eventos y retcon
context	Ensamblado del prompt por capas y presupuesto de tokens
quality	Validadores programáticos, judge y editor
guardrail	Palabras y temas prohibidos, normalización
policy	Policy engine y audit log

Feature	Qué contiene
process	Orquestador, worker, tabla de transiciones, hooks y roles
versioning	Versiones candidatas y publicadas, gate, regeneración dirigida, lectura y export
commons/	Base de datos, configuración, errores problem+json , cliente del modelo y observabilidad

La skill reutilizable `personalizacion-natural` vive en `backend/app/skills/` , fuera de las features. Las dependencias del diagrama son las principales; el test de arquitectura del repositorio comprueba que no hay ciclos.

A.3 · Bucle de generación de un capítulo

Cada capítulo pasa por Writer, los hooks de policy y de capítulo (en ese orden), Judge y policy engine; si algo falla, el Editor corrige o el Writer reescribe, y cada vuelta gasta un intento de un único contador.



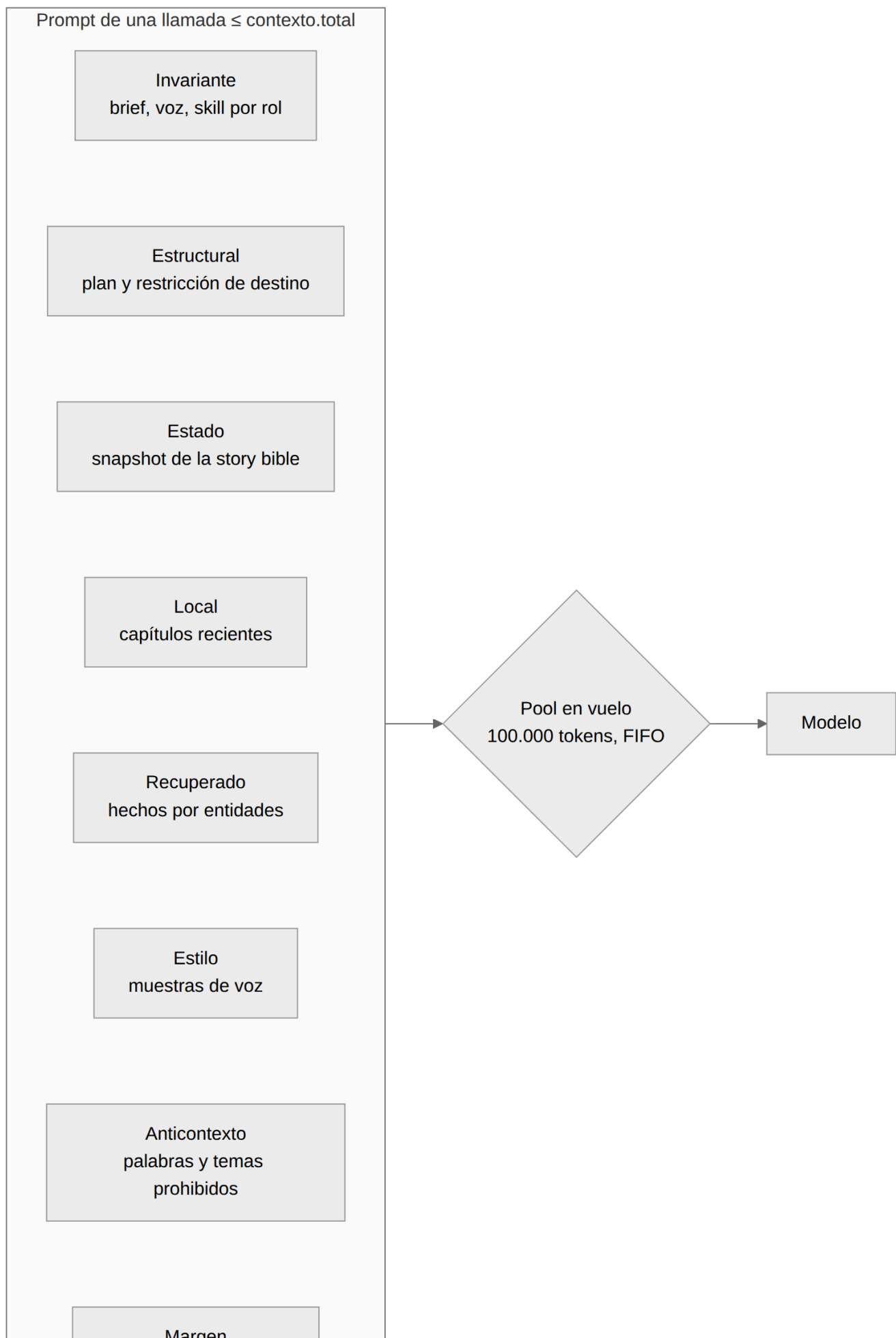
Rol	Modelo	Responsabilidad
Planner	Opus 5.5	Plan de la novela y restricciones de destino por capítulo
Writer	Opus 5.5	Escribe el capítulo con la skill <code>personalizacion-natural</code>
Editor	Opus 5.5	Corrige cuando un validador o el judge suspenden
Judge	Sonnet 5	Puntúa con la rúbrica; modelo distinto del Writer para evitar autopreferencia
Extractor	Sonnet 5	Extrae hechos, promesas y eventos del capítulo aceptado

Reglas del bucle:

- **Solo se consolida lo aceptado.** Un borrador rechazado no deja rastro en la story bible.
- **Un único contador por capítulo** (`max_intentos_capitulo`): correcciones del editor y reescrituras gastan del mismo; el guardrail añade un sublímite de reescrituras consecutivas.
- **Checkpoint tras cada capítulo aceptado:** si el proceso cae, se reanuda desde el último capítulo consolidado.
- **Chequeo incremental de Lean** tras aceptar cada capítulo, solo como aviso.

A.4 · Ensamblado del contexto y presupuesto de tokens

Cada prompt se construye con siete capas más un margen para la respuesta; si no cabe, se comprimen primero las capas menos esenciales, y la capa invariante nunca se recorta.





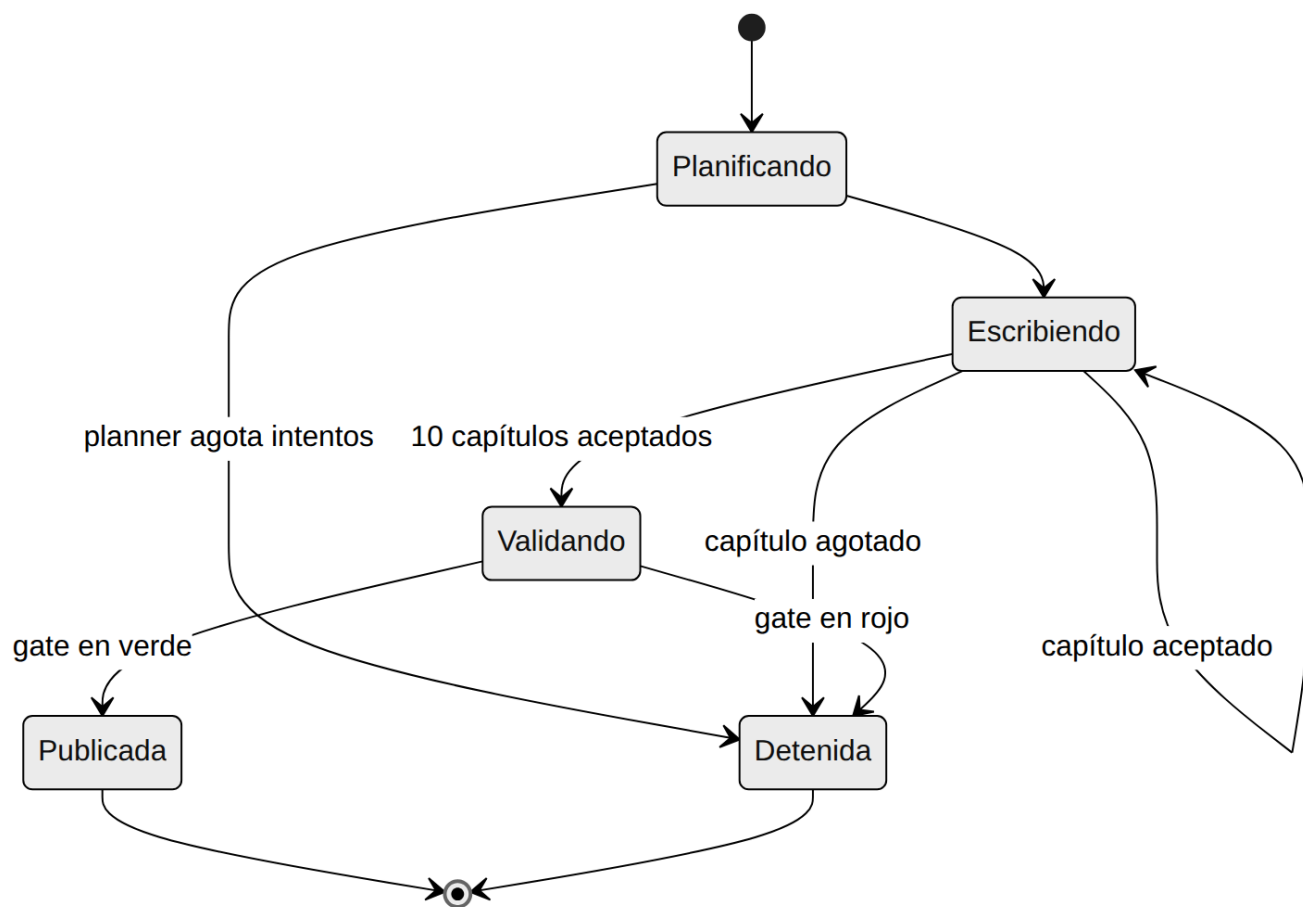
Regla	Cómo se aplica
Conteo antes de llamar	Se estima el prompt completo antes de enviarlo; si no cabe, falla en voz alta, nunca se trunca en silencio
Orden de degradación	Recuperado → Estilo → Local → Estado, parando en cuanto quepa
Lo que nunca se degrada	La capa invariante y la restricción de destino del capítulo
Invariante por rol	La skill <code>personalizacion-natural</code> solo se carga en Planner, Writer y Editor
Tokens concurrentes	Un pool de 100.000 tokens compartido por todas las llamadas en curso, con admisión FIFO; los capítulos van en serie y los validadores programáticos no consumen pool
Salida dentro del margen	El arranque comprueba que el margen es mayor o igual que el <code>max_tokens</code> de cada rol

Todas las cifras viven en `config/thresholds.yaml` ; ningún documento ni fichero de código las repite.

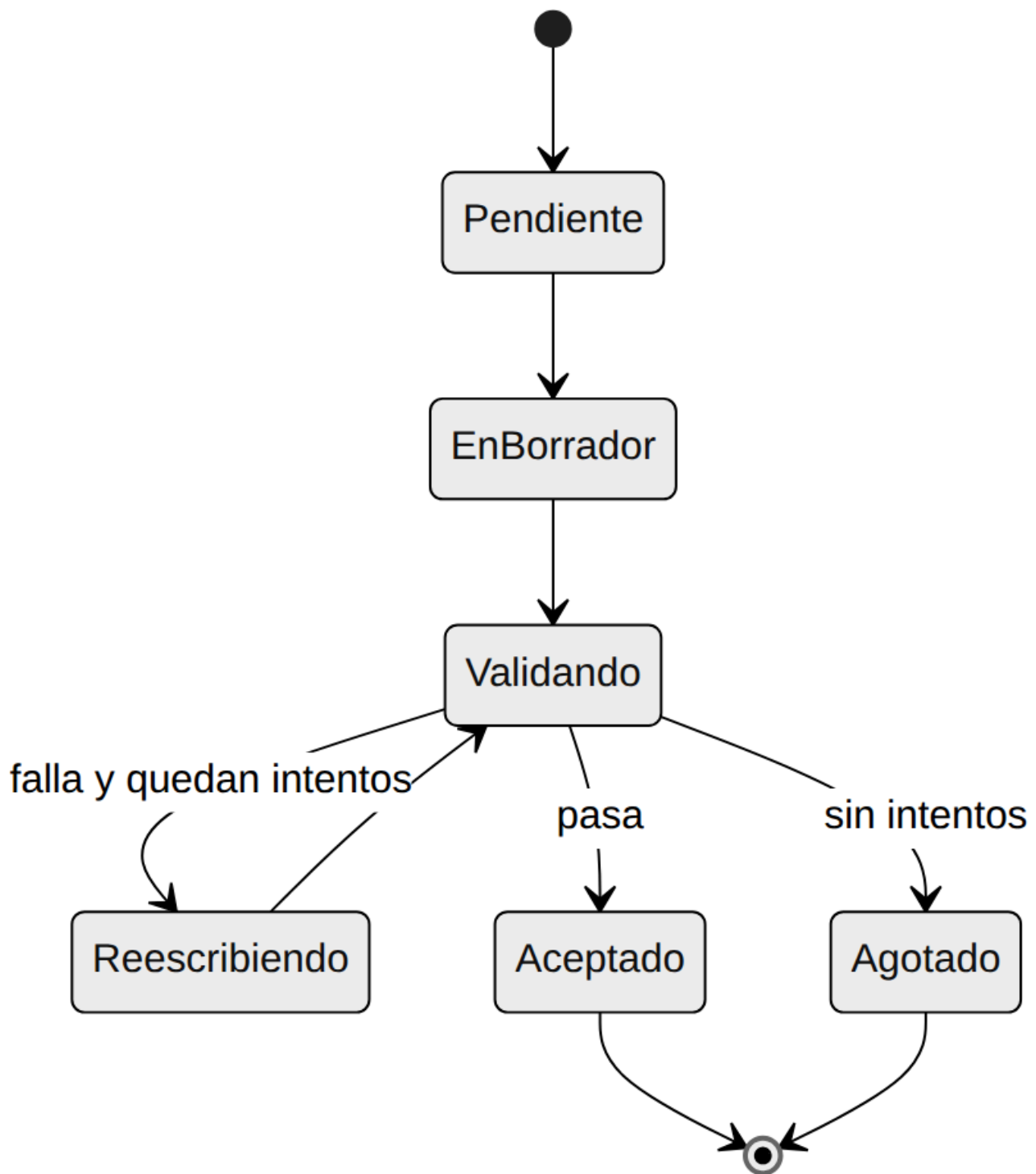
A.5 · Máquinas de estado

La generación, el capítulo y la versión tienen estados explícitos y persistidos; la tabla de transiciones del código es un dato que se comprueba contra la especificación TLA+.

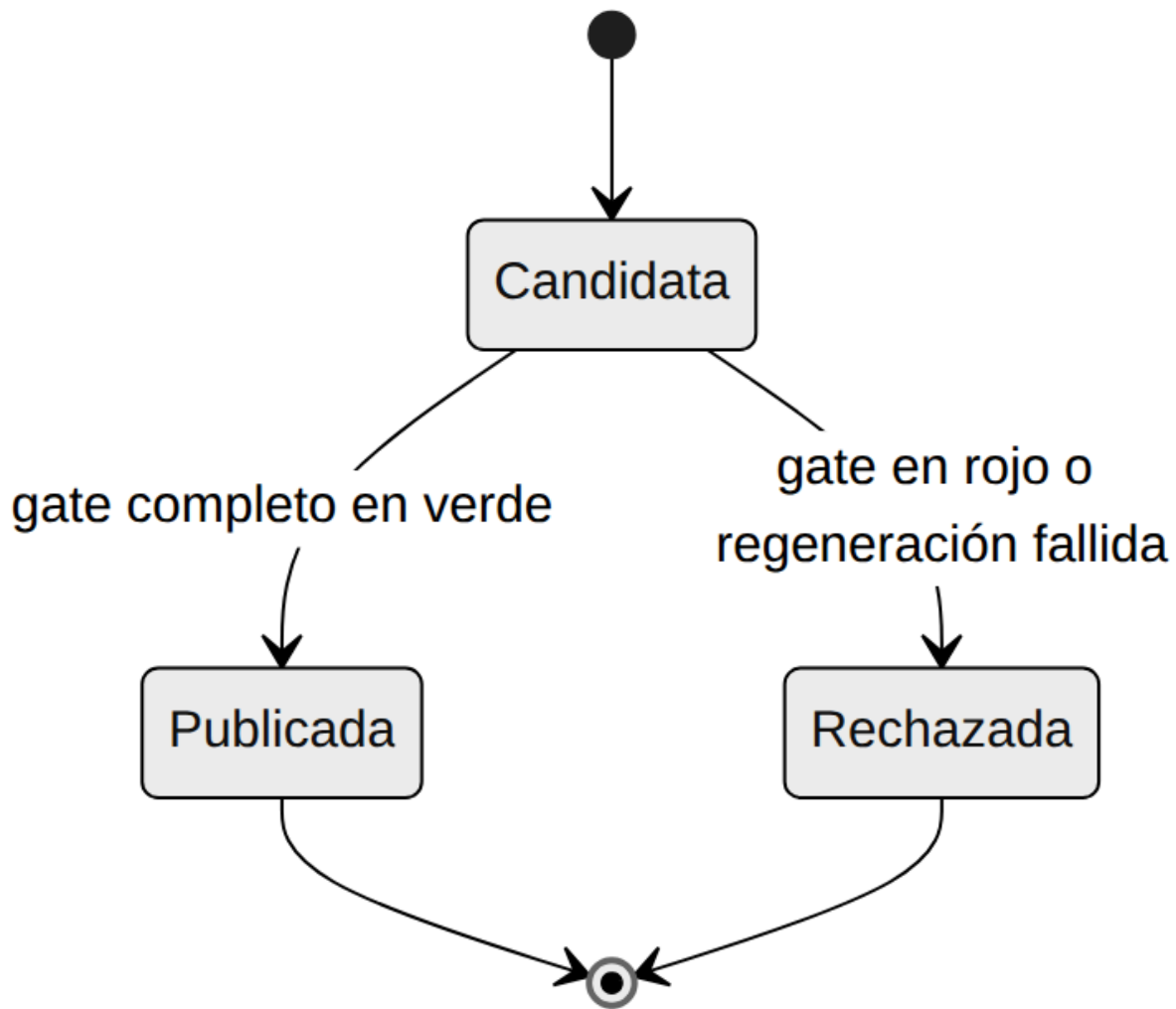
Generación de una novela



Capítulo dentro de una generación



Versión de una novela

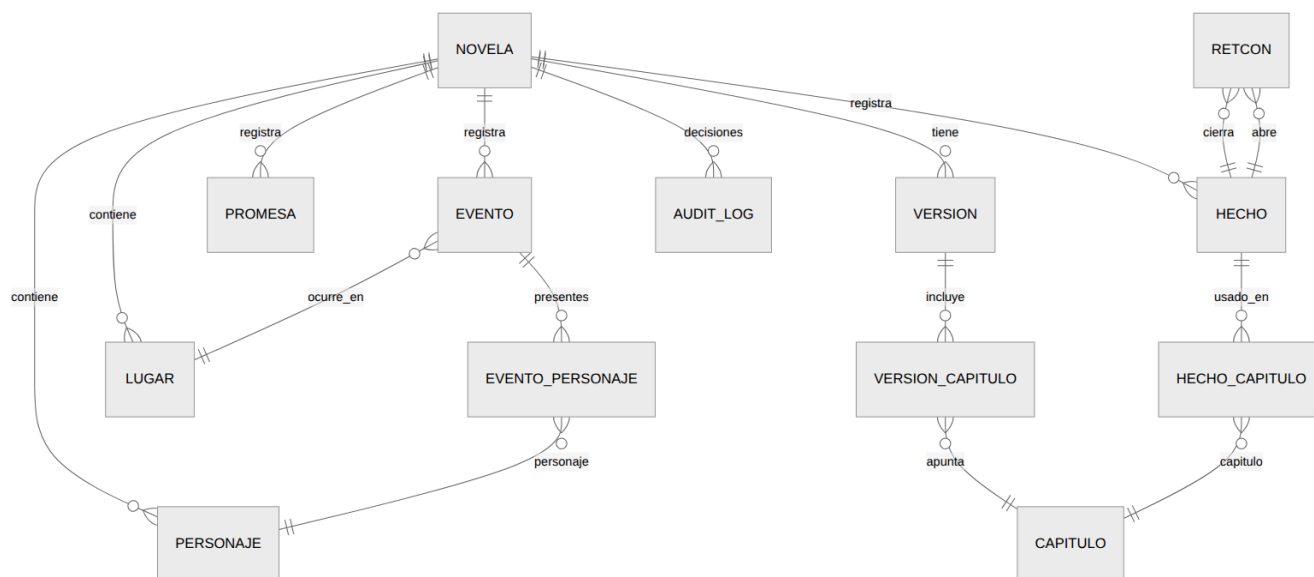


Invariante	Qué garantiza
Ninguna versión se publica sin pasar todos los validadores	Una candidata solo pasa a publicada con el gate en verde, <code>render_visual</code> y Lean incluidos
Una regeneración fallida no modifica ninguna versión publicada	La candidata queda rechazada; la novela sigue publicada con su versión vigente
La reanudación no duplica ni pierde capítulos	Los aceptados forman un prefijo contiguo; cada capítulo se consolida una vez por versión
Los reintentos nunca superan el límite	Un único contador por capítulo
Toda generación termina	En publicada o en detenida (liveness)

Los nombres de estados deben coincidir con `backend/app/process/transiciones.py`; revísalos antes de presentar.

A.6 · Story bible: esquema conceptual en SQLite

Cada hecho y cada evento tienen vigencia por versión, y la relación con los capítulos que los usan es la que permite regenerar solo lo afectado por un cambio.

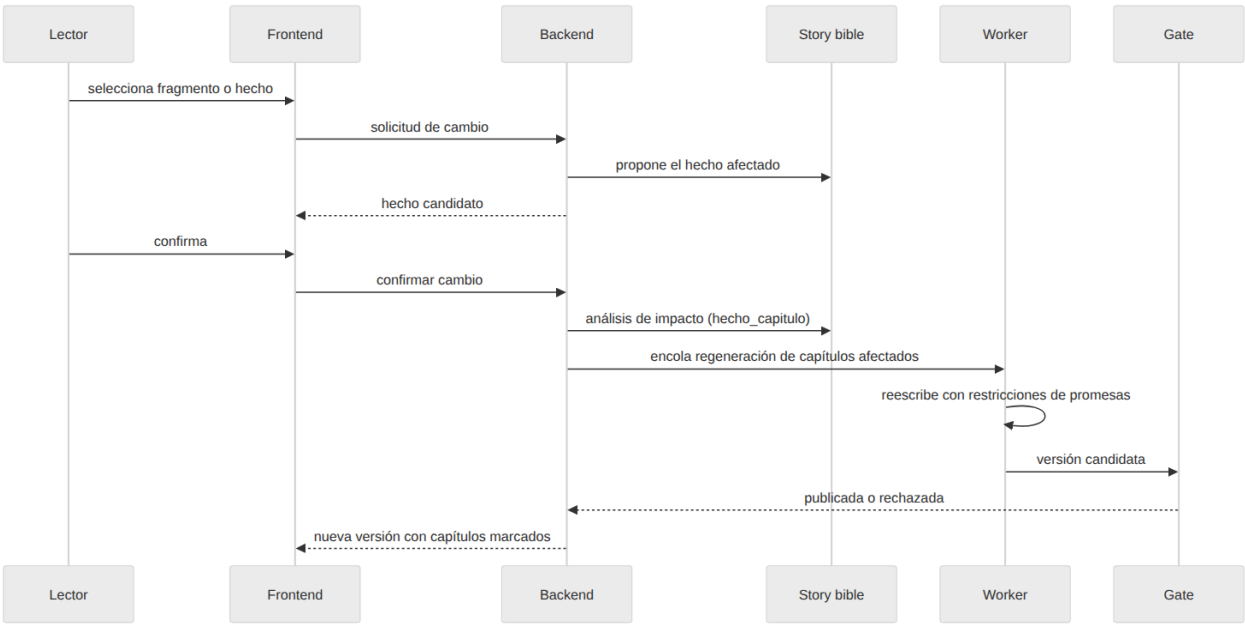


Entidad	Atributos clave	Para qué sirve
version	número correlativo, estado (candidata, publicada, rechazada), hash	Versiones inmutables; la anterior siempre consultable
hecho	vigencia version_desde / version_hasta	Consultar la story bible de cualquier versión
hecho_capitulo	hecho, capítulo, versión	Análisis de impacto: qué capítulos usan un hecho
promesa	apertura y pago vinculados a capítulos	Comprobar el cierre del arco (cierre_arco)
evento	momento narrativo, año opcional, versión	Cronología para Lean
evento_personaje	edad opcional	Invariantes de edad y de nacimiento
retcon	hecho antiguo → hecho nuevo	Linaje de los cambios del lector
audit_log	tipo de decisión, motivo	Registro del policy engine

Toda tabla de dominio lleva `novel_id`. Es un esquema conceptual: los nombres exactos de tablas y columnas están en las migraciones de `backend/app/commons/db/migrations/`.

A.7 · Regeneración dirigida por un cambio del lector

Un cambio del lector solo reescribe los capítulos que usan el hecho afectado, en una versión candidata; la versión vigente no se toca hasta que la nueva pasa el gate completo.

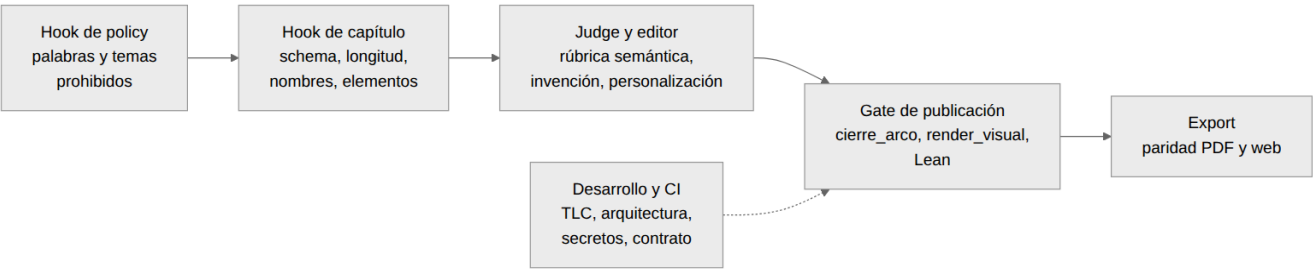


Paso	Detalle
Confirmación	El lector confirma el hecho propuesto: nunca se regenera por una inferencia silenciosa
Análisis de impacto	Solo los capítulos que usan el hecho en la versión vigente
Restricciones al reescribir	El capítulo conserva las promesas que abría y pagaba, y no abre promesas nuevas sin pago
Gate	Validadores, <code>cierre_arco</code> , <code>render_visual</code> y Lean sobre la candidata
Si falla	La candidata queda rechazada; la novela sigue publicada con la versión anterior
Resultado	Versión nueva con los capítulos modificados marcados; la anterior sigue consultable entera

En el ensayo de la demo, un cambio que afectaba solo al capítulo 7 se regeneró en 250 segundos por 0,53 USD nominales.

A.8 · Gate de publicación y puntos de ejecución de los validadores

Los validadores actúan en cinco puntos del flujo más el desarrollo; el gate de publicación es la última barrera y ninguna versión pasa a publicada sin superarlo entero.

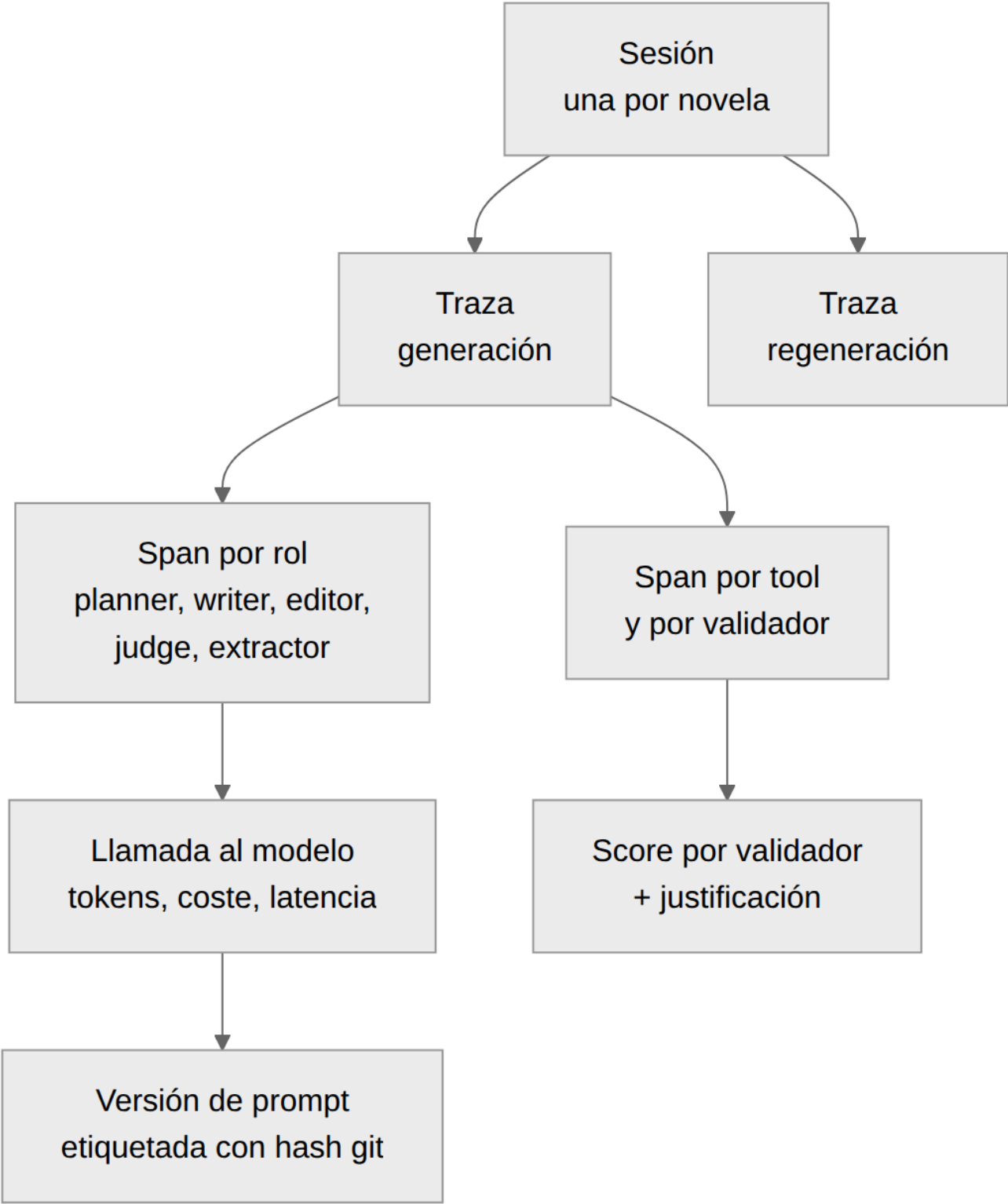


Punto de ejecución	Tipo	Ejemplos
Hook de capítulo	Programático	Schema de la salida, longitud, nombres exactos, elementos obligatorios
Hook de policy	Programático	Palabras prohibidas (tres niveles, normalizadas), temas excluidos
Judge y editor	Semántico (LLM-as-judge)	Continuidad, tono, arco, personajes, ritmo, personalización natural, invención sobre el destinatario
Gate de publicación	Programático y formal	<code>cierre_arco</code> , <code>render_visual</code> vía browser MCP, invariantes Lean (cronología, ubicación, edad, nacimiento)
Export	Programático	<code>paridad_pdf_web</code>
Desarrollo y CI	Formal y estático	TLC sobre la spec TLA+, test de correspondencia con la tabla de transiciones, contrato OpenAPI, secretos

Cada validador que corre en producción envía su resultado a Langfuse como score. La revisión humana usa la misma rúbrica que el judge para medir su acuerdo.

A.9 · Observabilidad en Langfuse

Cada novela es una sesión de Langfuse que agrupa su entrevista, su generación y sus regeneraciones; cada llamada al modelo queda vinculada a la versión exacta de su prompt.



Elemento	Qué registra
Sesión	Una por novela, con la entrevista, la generación y las regeneraciones
Traza	Una por generación o regeneración, con el entorno (dev , demo)

Elemento	Qué registra
Span	Uno por rol, por tool y por validador, con nombre identificable
Llamada al modelo	Modelo, tokens de entrada y salida, coste y latencia
Score	Uno por validador; los semánticos incluyen la justificación del judge
Versión de prompt	Los prompts viven en el repo y se sincronizan con Langfuse etiquetados con el hash de git

Con el proveedor `claude_code`, los tokens y el coste registrados son nominales, calculados con los precios de la API.